MODOLUME

Roteiro de apresentação
Documentação, PRDs e sistema

Material de apoio para apresentação ao professor | 06/11/2026

Base da análise: commit **810bc46**, branch **feat/backend-guilherme**. Este PDF reúne o roteiro e as informações discutidas na revisão. Não representa uma nova homologação nem alteração do código.

1. Abertura: o que é o projeto

> “O Modolume é um sistema educacional que conecta a prática de matemática do aluno ao acompanhamento do professor. O aluno utiliza um aplicativo com missões, atividades e gamificação. A equipe da escola utiliza um painel web para administrar usuários e turmas, acompanhar resultados, planejar aulas e distribuir atividades. O objetivo é transformar essa prática em informações que apoiem decisões pedagógicas. A proposta ainda precisa ser validada com usuários reais.”

Como usar este roteiro

Os blocos entre aspas são sugestões de fala. As tabelas servem como apoio para responder perguntas. Abra os documentos e as telas correspondentes enquanto explica; evite apenas ler todos os PRDs.

Ordem	Conteúdo
1	Visão geral e organização da documentação
2	Estrutura dos PRDs e exemplo de rastreabilidade
3	Arquitetura, stacks e volume de código
4	Logins, dados e integração
5	Percentual estimado e qualidade das histórias de usuário

Parecer que deve acompanhar a apresentação

MVP em desenvolvimento e homologação. Existem funcionalidades demonstráveis e testes aprovados, mas permanecem bloqueios de build, limitações de sincronização e validações práticas pendentes. “Implementado”, “testado” e “validado com usuários” são estados diferentes.

2 / Organização da documentação

> “A documentação está organizada por finalidade. Primeiro temos a visão do produto; depois, os requisitos de cada módulo; em seguida, decisões técnicas, diretrizes de interface e materiais de desenvolvimento. Essa divisão permite encontrar o que o sistema deve fazer e as razões das escolhas técnicas.”

Pasta	Finalidade
00-modelos	Modelos para manter a documentação padronizada.
01-produto	PRD geral: problema, público, objetivos, escopo e fases.
02-modulos	Os 17 PRDs que detalham as partes do sistema.
03-decisoes	ADRs: decisões de arquitetura e suas justificativas.
04-ux	Diretrizes de interface e usabilidade.
05-referencia	Contexto do projeto e modelo de dados.
06-desenvolvimento	Orientações técnicas, testes, auditorias e planos de ação.
07-historico	Registros históricos das entregas.
99-arquivo	Documentos substituídos, preservados para consulta.

Por que PRDs de módulo?

> “A ADR 0014 registra a decisão de substituir a SRS centralizada por PRDs de módulo, conforme a orientação recebida do professor. Os identificadores dos requisitos foram preservados para manter a rastreabilidade.”

O papel de cada documento

PRD geral: explica o produto e o problema. **PRD de módulo:** detalha o comportamento esperado. **ADR:** registra uma decisão técnica e suas razões. **Relatório de desenvolvimento:** registra o trabalho realizado. **Auditoria:** compara a proposta com as evidências atuais.

Os links internos da documentação usam a sintaxe [[nome do documento]], do Obsidian. Prepare essa navegação antes da apresentação para evitar links que não abrem no visualizador escolhido.

3 / Estrutura dos PRDs

> “Cada PRD começa explicando por que aquele módulo existe e para quem ele foi pensado. Depois descreve os comportamentos esperados, as histórias de usuário e as condições necessárias para aceitar a entrega. A história expressa a necessidade do usuário; o critério de aceite transforma essa necessidade em algo observável.”

Seção	Seção
1. Resumo executivo	7. Critérios de aceite
2. Problema e oportunidade	8. MVP e fases
3. Usuários e necessidades	9. Diretrizes do produto
4. Objetivos e critérios de sucesso	10. Dependências e riscos
5. Escopo, requisitos e regras	11. Pendências para aprovação
6. Histórias de usuário	12. Documentos vinculados

Campo	Legenda
Prioridade	E: essencial; I: importante; D: desejável.
Verificação	T: teste; D: demonstração; N: inspeção; A: análise ou medição.
Situação	Implementado, parcial, pendente ou a medir.

Exemplo para demonstrar ao professor

> “No PRD-05, o requisito MOD-GAM-002 determina a atribuição de pontos. A história HU-GAM-01 explica que o aluno quer perceber seu avanço pela constelação. O critério HU-GAM-01 / CA-01 estabelece que uma missão de cinco fases, com quatro acertos de primeira, gera 12 pontos: 5 + 4 + 3. Podemos demonstrar o comportamento no aplicativo e conferir o teste da regra.”

Cadeia: problema → requisito → história → critério de aceite → implementação → evidência.

Use uma missão fora do bônus do desafio diário. O exemplo comprova uma regra local; não comprova a consolidação de pontos entre aparelhos. Números como CA-01 se repetem entre histórias: cite sempre também a HU.

4 / Arquitetura e stacks

> “O projeto usa um monorepositório: aplicações diferentes ficam no mesmo repositório e compartilham um núcleo de regras. Temos quatro partes principais: aplicativo mobile, aplicação web, API e núcleo compartilhado.”

Parte / local	Stack declarada	Responsabilidade
Mobile apps/mobile	React Native 0.86.3; React 19.2.3; Expo 57; Expo Router; TypeScript	Experiência do aluno, missões, perfil e dados locais.
Web apps/web	Next.js 15; React 19; Tailwind CSS 4; TypeScript	Site e painéis da equipe escolar.
API apps/api	NestJS 11; Prisma 6; Zod; JWT; bcrypt	Autenticação, autorização, validação e persistência.
Núcleo packages/core	TypeScript; Zod; Vitest	Regras, cálculos, tipos e conteúdos compartilhados.
Banco	PostgreSQL	Dados institucionais e registros persistentes.
Organização	pnpm; Turborepo	Dependências e execução das tarefas dos pacotes.

As versões representam os manifests do projeto. Dependências declaradas com intervalos podem ter versões específicas diferentes no lockfile.

Como as partes se relacionam

> “O mobile e o painel se comunicam com a API, que controla o acesso ao banco. O core é compartilhado para reduzir duplicação das regras. Porém, compartilhar código não garante que todas as telas usem os mesmos indicadores corretamente; essa integração também precisa ser verificada.”

Fluxo de prática: aluno → mobile → API → PostgreSQL → API → painel do professor.
Fluxo de atividade: professor → painel → API → PostgreSQL → API → mobile.

5 / Volume de código

> “Nesse recorte, o projeto possui aproximadamente 46 mil linhas distribuídas em 230 arquivos, com média de 201 linhas por arquivo. O aplicativo mobile concentra o maior volume. Essa contagem representa tamanho, não qualidade nem percentual de conclusão.”

Parte	Arquivos	Linhas	Média por arquivo
Mobile	87	23.311	268
Web	103	13.664	133
API	21	5.048	240
Core	19	4.166	219
Total	230	46.189	201

Como foi contado

Foram contadas linhas físicas, incluindo comentários e linhas vazias, nos arquivos encontrados em apps e packages com extensões .ts, .tsx, .js, .mjs, .cjs, .css, .prisma e .sql. O recorte inclui testes e configurações nessas extensões. Exclui dependências, builds ignoradas, documentação e arquivos .d.ts.

A média não descreve todos os arquivos

O arquivo packages/core/src/missoes.ts possui 1.677 linhas, incluindo conteúdo das missões. O editor web de atividades possui 796 linhas. Arquivos grandes merecem avaliação de responsabilidades, mas o tamanho, isoladamente, não comprova um problema.

Como responder se perguntarem sobre qualidade

> “A qualidade é avaliada pelo comportamento correto, manutenção, testes, segurança e experiência do usuário. O número de linhas ajuda a mostrar a dimensão do projeto, mas não é nosso critério de qualidade.”

6 / Logins e controle de acesso

> “Existem dois fluxos principais de autenticação: um para a equipe escolar e outro para o aluno. A permissão depende do perfil, da instituição e dos vínculos. Estar autenticado não significa poder acessar qualquer turma.”

Público / condição	Funcionamento atual
Equipe escolar	Professor, coordenação e administrador entram com e-mail e senha. A API verifica o hash com bcrypt e emite JWT com validade de oito horas.
Senha provisória	A pessoa precisa trocar a senha para acessar as operações protegidas pelo bloqueio de senha provisória.
Aluno	Utiliza código pessoal de oito caracteres e PIN de quatro números.
Primeiro acesso do aluno	Se não existe PIN, o aluno define um. O servidor rejeita PINs considerados fracos.
Erros de PIN	Cinco erros consecutivos geram bloqueio de 15 minutos.
Sessão do aluno	Token com validade configurada de 180 dias; o servidor também verifica condições de acesso e versão da credencial.

Onde fica a sessão

Web: cookie assinado e HttpOnly, que não pode ser lido pelo JavaScript da página. Com a API ligada, a sessão também contém o token usado pelo servidor web.

Mobile: sessão e dados locais persistidos com AsyncStorage. O isolamento por aluno foi implementado recentemente. Esse armazenamento não deve ser apresentado como um cofre criptografado.

Recuperação de acesso

Não apresente recuperação automática de senha por e-mail como concluída. O fluxo atual orienta procurar a escola, e há redefinição administrativa com senha provisória. No acesso do aluno, existe redefinição de credenciais conforme as permissões da equipe.

7 / Dados e integração

> “O aplicativo registra a prática localmente e possui uma fila para envio ao servidor. Com conexão e sessão válida, envia os dados à API. A API valida o acesso e grava no PostgreSQL. O painel consulta os dados disponíveis. No sentido contrário, o professor cria atividades e o aplicativo consulta as que são destinadas ao aluno.”

Integração	Situação
Web → API → banco	Implementada, com fluxos administrativos e pedagógicos.
Mobile → API	Implementada para autenticação, resultados, progresso e consulta de atividades.
Uso local / offline	Implementado; sincronização ainda precisa de consolidação.
Isolamento por aluno	Correção implementada e testes de auxiliares aprovados; falta validação completa em dispositivo.
Múltiplos aparelhos	Parcial: consolidação de ganhos e compras apresenta problemas.
Atividades → indicadores	Parcial: os caminhos de registro não alimentam todas as superfícies igualmente.
Avaliação, feedback e notificações	Não devem ser apresentados como um ciclo completo entregue.
Entrega de produção	Bloqueada pela falha de build web e pendências de estabilidade.

Como comprovar integração na apresentação

O painel possui modo de demonstração quando API_URL não está configurada. Uma tela preenchida, sozinha, não comprova integração com o banco. Crie um registro e acompanhe sua persistência e leitura em outro ponto do sistema.

No mobile, a configuração utiliza EXPO_PUBLIC_API_URL. O endereço precisa ser acessível pelo aparelho; localhost no telefone se refere ao próprio telefone.

8 / Percentual e situação atual

Estimativa para comunicação: aproximadamente 70% de avanço funcional. É um julgamento técnico sobre o escopo documentado, não uma medição oficial nem um percentual de prontidão para produção.

> “Estimamos cerca de 70% de avanço funcional do escopo documentado. O sistema está em desenvolvimento e homologação. Essa estimativa não significa 70% de prontidão para produção: ainda temos bloqueios de build, sincronização e validação prática.”

O que sustenta essa estimativa

Os componentes principais e diversos fluxos já existem. Na revisão anterior passaram 122 testes. Entretanto, existem módulos pendentes, integrações parciais e falhas importantes. A build web não concluiu e a validação com usuários reais permanece pendente.

Evidência da revisão	Resultado
API	63 testes passaram.
Core	43 testes passaram.
Mobile	16 testes passaram, com dependências simuladas.
Typecheck / lint mobile	Passaram.
Lint geral	Web solicita configuração interativa.
Build web	Falha no prerender de /404 e /_error, envolvendo useRef.

Percentual documental não é conclusão comprovada

Os documentos classificam 166 de 223 requisitos ativos como implementados, aproximadamente **74,4%**. Os demais são 23 parciais, 19 pendentes e 15 a medir. Os requisitos têm pesos diferentes e alguns status estão desatualizados. Essa contagem não comprova atendimento integral.

Não se atribui aqui um percentual preciso à prontidão de produção: faltam critérios formais de liberação e evidências para cada um. A homologação em PostgreSQL e a tentativa em iPhone descritas pela equipe não foram reproduzidas na revisão.

9 / Histórias de usuário

Os 17 PRDs de módulo possuem histórias e seções de critérios de aceite. Foram identificadas 103 histórias ativas e uma substituída no PRD-03. Não é correto afirmar que todas estão corrigidas, aprovadas e comprovadas no sistema.

> “Todos os módulos possuem histórias de usuário vinculadas a requisitos e critérios de aceite. Estamos revisando a precisão dos perfis, os benefícios esperados e a correspondência dos status com o código. A existência da história não significa que ela já foi implementada ou validada.”

Exemplo	Ajuste necessário
PRD-03: “Como gestão…”	Especificar administrador ou coordenação conforme a permissão real.
PRD-07: troca de senha	Atualizar o status para acompanhar a implementação.
PRD-10: outro aparelho	Explicitar benefício, limites e cenários de sincronização.
PRD-14: “quem está fazendo”	Alinhar requisito e aceite com a disponibilidade real do estado iniciado.
PRD-17: desempenho e uso	Definir condições de medição, dispositivos e critérios objetivos.

Exemplo de melhoria de redação

Atual: “Como aluno, quero recuperar meu progresso em outro aparelho.”

Sugestão: “Como aluno, quero recuperar meu progresso ao entrar em outro aparelho, para continuar meus estudos sem perder conquistas e atividades realizadas.”

A redação melhora a história, mas não corrige a implementação. São necessários critérios para ganhos concorrentes, duplicação de envios, compras e recuperação do histórico. Histórias com objetivos distintos, como desligar vibração e apagar progresso, podem ser separadas para facilitar priorização e teste.

As sugestões deste roteiro não alteraram os PRDs. Ausência da palavra “para” não torna uma história automaticamente inválida; o importante é que ator, necessidade e valor estejam claros e que o aceite seja verificável.

10 / Preparação para apresentar

Tempo	Demonstração sugerida
0–2 min	Problema, público e visão geral do produto.
2–4 min	Pastas da documentação, índice e ADR 0014.
4–7 min	PRD-05: requisito, história, aceite, regra e demonstração.
7–10 min	Arquitetura, login e criação/consulta de uma atividade.
10–12 min	Dados reais versus demonstração; integração e limites.
12–15 min	Percentual estimado, pendências e alinhamento com o professor.

Checklist do ensaio

1. Abrir PRD geral, índice e PRDs escolhidos nos trechos corretos.
2. Preparar contas e dados fictícios para demonstração.
3. Confirmar API e conexão do aparelho antes de começar.
4. Ensaiar um cenário permitido e um cenário recusado.
5. Levar evidência dos testes, identificada pelo commit.
6. Manter uma gravação de apoio, identificada como gravação.
7. Explicar pendências sem apresentar metas como resultados medidos.

Perguntas para alinhar com o professor

A divisão em módulos atende ao formato solicitado? Quais módulos são obrigatórios nesta entrega? Quais metas, amostras e instrumentos serão aceitos para validar usabilidade e resultados pedagógicos? Quem revisa e aprova os PRDs?

Referências utilizadas

Repositório local Modolume, commit 810bc46: docs/README.md; PRD geral; PRDs 01 a 17; ADR 0014; package.json da raiz e dos quatro pacotes; apps/api/src/acesso.ts; apps/api/prisma/schema.prisma; arquivos de sessão web e estado mobile. Contagens realizadas nos arquivos locais.

Contexto complementar: Modolume_Guia_Ana_Git_Instalacao.md; Modolume_Relatorio_05-06_Nov_2026.docx; reverificação de 06/11/2026. As evidências executadas e as limitações foram preservadas neste roteiro.